Nullmask: Transact Privately from Any Multichain Wallet

Tomas Krnak

March 17, 2026

1 Introduction

Nullmask is a novel on-chain privacy architecture that enables users to transact privately from any existing multichain wallet. It requires no new wallets, seeds, backups, or special addresses—just a one-time setup that takes less than 20 seconds.

Instead of depending on wallets implementing new privacy features or users migrating to new wallets, Nullmask integrates seamlessly into any existing multichain wallet as a new *virtual* network. This network is not a real network like Arbitrum; it is simply a mechanism that allows us to insert a custom RPC proxy service between a wallet and a full RPC node. This proxy shields outgoing transactions and decrypts incoming transactions for wallet accounts. It can run either on a remote server or seamlessly in the background on a user's device (mobile app, browser extension).

Our design is only possible due to recent breakthroughs in zk-SNARKs. Specifically, the UltraHonk prover enables us to prove the integrity of an EIP-1559 transaction and verify its ECDSA signature on a consumer-grade device in approximately two seconds.

1.1 Nullmask Key Advantages

- **Unified UX**: Nullmask does not require any change in user behavior. Users can continue using the wallets and workflows to which they are already accustomed.
- **Ultimate Compatibility**: Nullmask does not require any code modifications in wallets. It simply installs as a new EVM network and already works in MetaMask. We expect that other privacy SDKs such as Kohaku will take years to achieve wider adoption.
- **Enhanced Security**: Spending authority never leaves the wallet; therefore, Nullmask is more resilient to phishing attacks (illustrated in Figure 2).
- **Hardware Wallet Friendliness:** Our design provides fully secure hardware wallet support without additional effort.

1.2 Challenges & Solutions

Challenge	Solution
The circuit that validates an Ethereum transaction has enormous complexity.	Using the most advanced zk-SNARK prover.
The proxy can replay virtual transactions.	Shielded Transaction Nonces (Section 6.1).
The proxy can produce unspendable notes.	Custom zk-verifiable note encryption scheme (Alg. 5.1).
The proxy can tamper with the 0x-to-zk address translation.	On-chain verifiable key registry.

2 Nullmask Architecture

Our architecture consists of several parties (see Figure 1):

- **User:** An end user utilizing their multichain wallet to send and receive transactions and swap tokens.

- **Wallet:** A standard wallet supporting the `personal_sign` method and the addition of custom networks with custom RPC servers.
- **Proxy:** An RPC service that runs either locally or remotely, providing the following functionalities:
  - Generates a *Viewing Key* and a *Receiving Key* and stores them locally.
  - Responds to wallet queries (such as `eth_sendTransaction` and `eth_getBalance`).
  - Shields outgoing transactions and forwards them to Relayers.
  - Scans the Contract for new transactions, trial-decrypts them, and maintains the wallet's shielded balances.
- **Relayers:** A decentralized network of servers that receive shielded transactions and relay them to the contract, paying L1 fees. They also conceal the proxies' IP addresses.
- **Contract:** An L1 contract behaving as a Privacy Pool with methods such as `deposit`, `withdraw`, and `shieldedTransfer`.

Figure 1: Nullmask Architecture

3 Security Objectives

Our privacy objective is as follows: all incoming and outgoing account transfers are completely private. No external observer can decrypt the fields of shielded transactions. Only the proxy that has access to the account's viewing key can inspect the account's transaction history.

Our security objective is as follows: every shielded action must be authorized by an in-wallet-confirmed (signed) virtual transaction. Even a malicious actor with access to the account's Viewing Key and transaction history cannot initiate shielded actions or manipulate users' funds.

4 Key Derivation

Similarly to other privacy protocols, Nullmask needs additional key material to realize private transfers. All these keys are derived deterministically from a wallet signature. A signature constitutes a strong source of entropy owing to the security of the underlying signature scheme. Furthermore, since all modern wallets use deterministic signatures defined by RFC 6979, all keys can be recovered by re-signing a fixed message.

The Nullmask *Viewing Key* serves for shielding and de-shielding transactions. It grants view access to the account transaction history. The Nullmask *Receiving Key* serves for receiving funds.

Users do not have to know or store either of these two keys. The Viewing Key and the Receiving Key are generated by the proxy service. The Viewing Key remains stored there while the Receiving Key is posted on-chain.

In practice, we employ a zk-SNARK-friendly curve such as Baby Jubjub, BN254, or Pasta. We denote this curve by $\mathbb{E}$ and its scalar field by $\mathbb{F}$.

definition 1: Nullmask *Viewing Key* is a tuple $(\mathtt{pk}, \mathtt{nk}, \mathtt{ivk}, \mathtt{ovk})$ where:

- $\mathtt{pk}$: Public Key, a secp256k1 point corresponding to the wallet account's public key.
- $\mathtt{nk} \in \mathbb{F}$: Nullifying Key, used for the derivation of deterministic note nullifiers.
- $\mathtt{ivk} \in \mathbb{F}$: Incoming Viewing Key, used for trial decryption of incoming notes.
- $\mathtt{ovk}$: Outgoing Viewing Key, a $\mathbb{E}$ scalar serving to encrypt receipts of the notes sent from the account.

definition 2: The Nullmask `EXPORT_VK_MESSAGE` is a special protocol-fixed message. The signature of this message serves as an entropy source for all other Nullmask keys. In the current version of the protocol we use: *Authorize view-only access to Nullmask shielded account.*

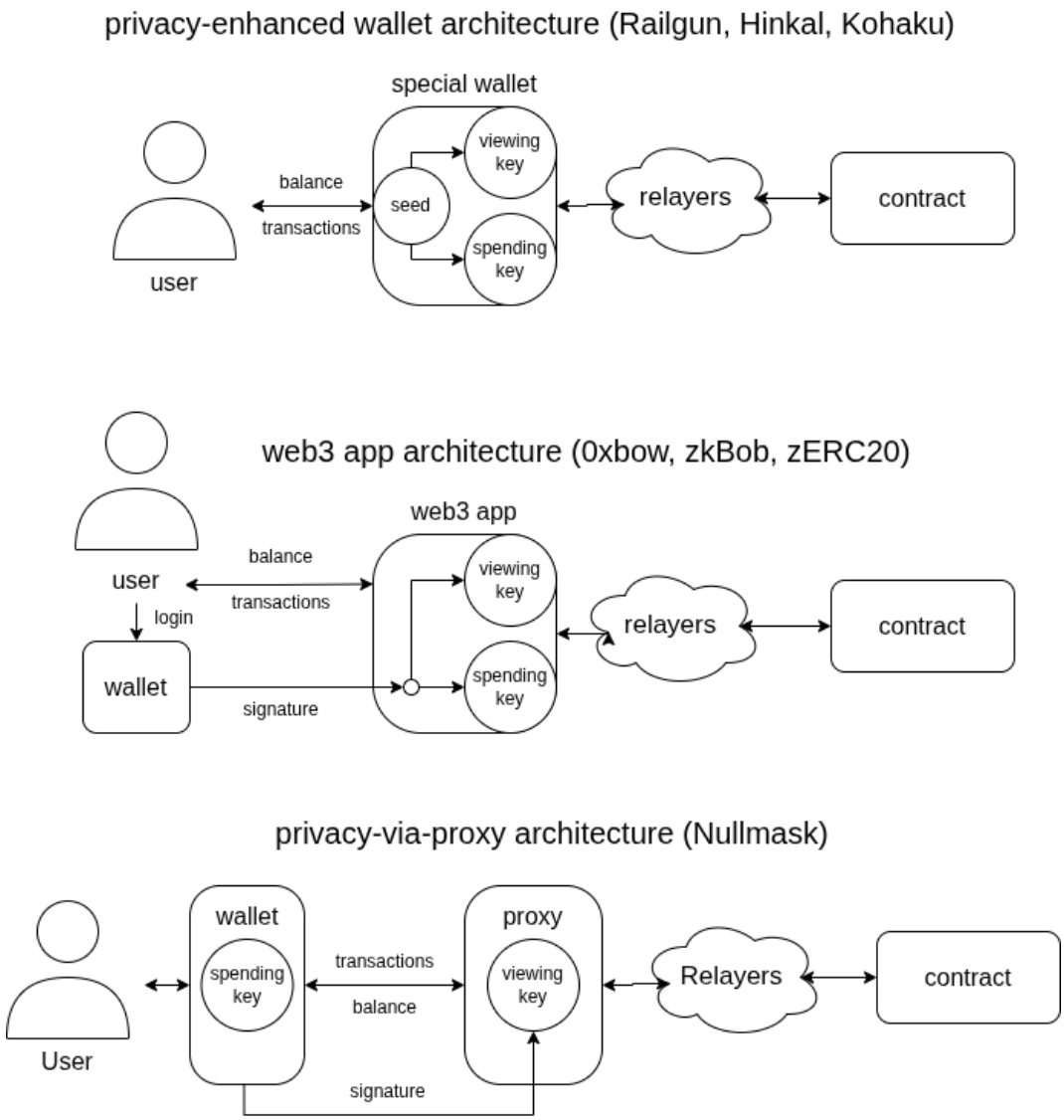

Figure 2: Architecture Comparison

Algorithm 1: Derive Viewing Key

Input: wallet connection
Output: A Viewing Key $(\texttt{pk}, \texttt{nk}, \texttt{ivk}, \texttt{ovk})$

1. Request a `personal_sign` of the message `EXPORT_VK_MESSAGE`.
2. Recover `pk` from `signature`.
3. $\texttt{seed} \leftarrow \text{Poseidon2T4}(\texttt{signature})$.
4. $\texttt{nk} \leftarrow \text{Poseidon2T4}(\texttt{seed}, 1)$.
5. $\texttt{ivk} \leftarrow \text{Poseidon2T4}(\texttt{seed}, 2)$.
6. $\texttt{ovk} \leftarrow \text{Poseidon2T4}(\texttt{seed}, 3)$.
7. Return $(\texttt{pk}, \texttt{nk}, \texttt{ivk}, \texttt{ovk})$.

definition 3: Nullmask *Receiving Key* is a triple $(\texttt{pk}, \texttt{pnk}, \texttt{ek})$ where:

- `pk`: Public Key, a secp256k1 point corresponding to the wallet account's public key.
- $\texttt{pnk} \in \mathbb{F}$: Public Nullifying Key, defined as the hash of `nk`.
- `ek`: Encryption Key, an $\mathbb{E}$ point used for encrypting notes destined for this recipient.

Algorithm 2: Derive Receiving Key

Input: A Viewing Key $(\texttt{pk}, \texttt{nk}, \texttt{ivk}, \texttt{ovk})$
Output: The corresponding Receiving Key $(\texttt{pk}, \texttt{pnk}, \texttt{ek})$

1. $\texttt{pnk} \leftarrow \text{Poseidon2T4}(\texttt{nk})$.
2. $\texttt{ek} \leftarrow \texttt{ivk} \cdot \mathsf{G}$.
3. Return $(\texttt{pk}, \texttt{pnk}, \texttt{ek})$.

Algorithm 3: Hash Receiving Key

Input: Receiving Key $(\texttt{pk}, \texttt{pnk}, \texttt{ek})$
Output: Receiving Key hash `rk_hash`

1. $\texttt{address} \leftarrow \texttt{keccak}(\texttt{pk})[12:]$.
2. $\texttt{buffer} \leftarrow \texttt{pk}$ (64 bytes) $\|$ $\texttt{address}$ (20 bytes).
3. Pad `buffer` with 9 zero bytes to length 93 bytes.
4. Split `buffer` into 3 parts of 31 bytes each.
5. Return $\text{Poseidon2T4}(\texttt{part1}, \texttt{part2}, \texttt{part3}, \texttt{pnk}, \texttt{ek.x}, \texttt{ek.y})$.

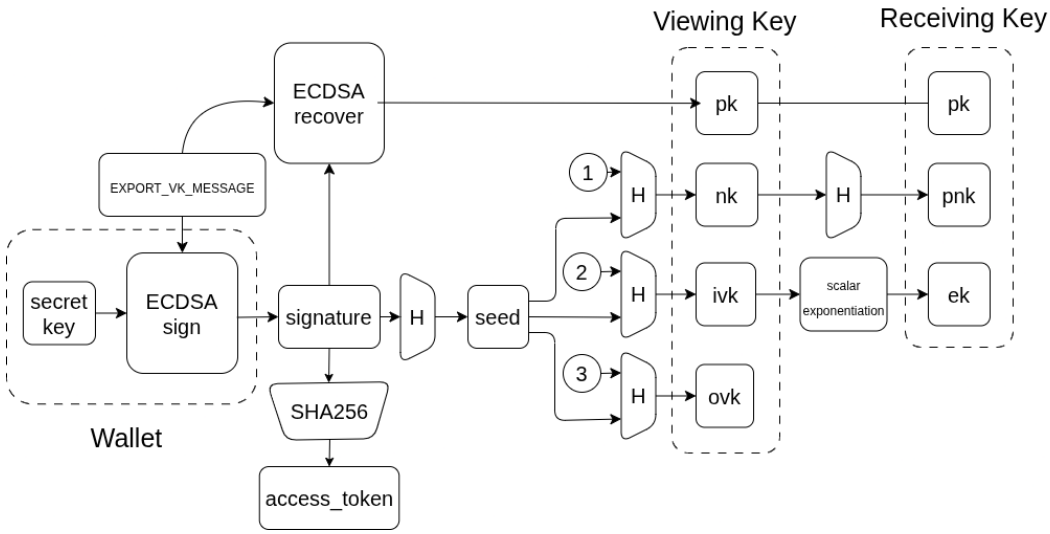

Figure 3: Key Derivation: `H` denotes the Poseidon2T4 hash function.

5 Notes (Encrypted UTXOs)

We employ a UTXO model for the shielded account state, referring to each UTXO as a *Note*.

definition 4: A Nullmask *Note* is a tuple ($\texttt{rk_hash}, \texttt{value}, \texttt{coin_id}, \texttt{rk_trapdoor}, \texttt{value_trapdoor}, \texttt{nfs_hash}$) where:

- `rk_hash`: hash of the Receiving Key that owns the note.
- `value`: note value.
- `coin_id`: token address (zero address for native ETH).
- `rk_trapdoor`: masking factor of note owner.
- `value_trapdoor`: masking factor of note value.
- `nfs_hash`: hash of all nullifiers from the shielded action that created this note.

For notes created via deposits to the Nullmask contract, `nfs_hash` is set to a unique deposit index. Including the hash of all nullifiers of the shielded action that created the note serves three purposes:

- **Uniqueness:** It guarantees note commitment and nullifier uniqueness and prevents faerie gold attacks.
- **Recursion:** It enables recursive proving—a shielded action proof can verify proofs of the shielded actions that funded it.
- **Compliance:** It enables Private Proof of Innocence and other compliance mechanisms.

Algorithm 4: Note Commitment

Input: A note ($\texttt{rk_hash}, \texttt{value}, \texttt{coin_id}, \texttt{rk_trapdoor}, \texttt{value_trapdoor}, \texttt{nfs_hash}$)
Output: Note commitment `cm`

1. $\texttt{rk_commitment} \leftarrow \text{Poseidon2T4}(\texttt{rk_hash}, \texttt{rk_trapdoor})$
2. $\texttt{value_commitment} \leftarrow \text{Poseidon2T4}(\texttt{value}, \texttt{coin_id}, \texttt{value_trapdoor})$
3. Return $\text{Poseidon2T4}(\texttt{rk_commitment}, \texttt{value_commitment}, \texttt{nfs_hash})$

definition 5: For a note with commitment `cm` belonging to a Viewing Key whose nullifying key is `nk`, the *note nullifier* is computed as:

$$\texttt{nf} := \text{Poseidon2T4}(\texttt{cm}, \texttt{nk})$$

5.1 Note Encryption

We employ standard key encapsulation for the symmetric encryption of notes. The note encryption key is derived via a Diffie–Hellman key exchange using the recipient's receiving key and an ephemeral note key. The note sender also attaches an encrypted outgoing viewing key `ek_out` to enable recovery of the history of outgoing transfers.

Algorithm 5: Encrypt Note

Input:

- A Note `note` whose commitment is `cm`,
- Recipient's Encryption Key `ek`, and
- Sender's Outgoing Viewing Key `ovk`.

Output: Encrypted Note $C_{\texttt{note}}$.

1. $\texttt{epk} \leftarrow \text{randomScalar}()$
2. $\texttt{dhek} \leftarrow \texttt{epk} \cdot G$
3. $\texttt{shared_secret} \leftarrow \text{Poseidon2T4}(\texttt{epk} \cdot \texttt{ek})$
4. $(m_1, m_2, m_3) \leftarrow \text{Poseidon2T4}(\texttt{shared_secret}, 1)$
5. $(m_4, m_5, \texttt{tag}) \leftarrow \text{Poseidon2T4}(\texttt{shared_secret}, 2)$
6. $\texttt{ciphertext} \leftarrow \texttt{note} + (m_1, m_2, m_3, m_4, m_5)$
7. $\texttt{ek_out} \leftarrow \texttt{shared_secret} + \text{Poseidon2T4}(\texttt{cm}, \texttt{ovk})$
8. Return $(\texttt{dhek}, \texttt{ek_out}, \texttt{tag}, \texttt{ciphertext})$

For the sake of circuit constraint optimization, we use the Poseidon2T4 hash function to generate the encryption mask. We compute $\text{Poseidon2T4}(\texttt{shared_secret}, i)$ for $i = 1, 2$, obtaining three masking

elements per iteration and six masking elements in total. Only the first five `note` fields are masked; `nfs_hash` is a public value. The last masking element is retained as `tag` and serves as a verification reference for successful trial decryption.

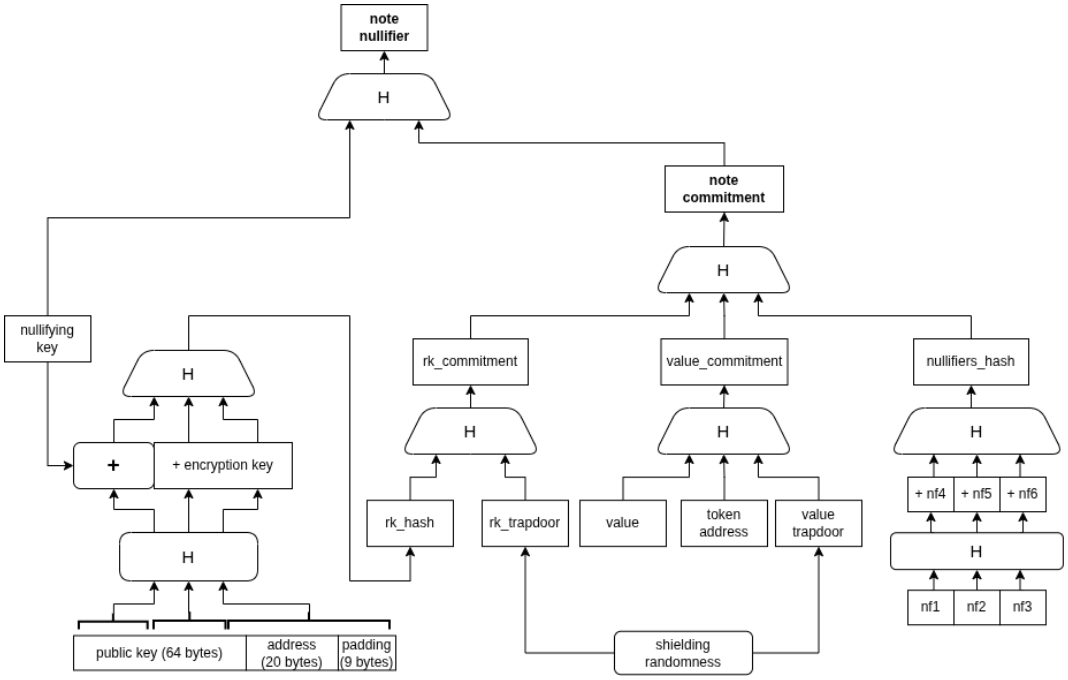

Figure 4: Note nullifier and commitment derivation. `H` denotes the Poseidon2T4 hash function.

6 Shielded Transfers

The proxy translates an EIP-1559 account-model transaction into its shielded UTXO-model equivalent. In the current design, each shielded action is funded by 1–6 notes and produces 1–3 new notes: an outgoing token transfer note for the transaction recipient, a change token note for the sender, and a change gas note for the sender.

6.1 Transaction Nullifiers

Nullmask shielded actions must be authorized by signed virtual transactions. This "second-factor authorization" differs from Zcash and Railgun.

- Railgun and Zcash: The host chooses notes that fund a shielded transaction; the wallet signs shielded transaction data, which include note nullifiers.
- Nullmask: The wallet signs an intent for a transaction; the host (proxy) chooses notes to fund the shielded transaction.

This introduces a fundamental problem: a malicious proxy could replay the transaction intent.

We use *Transaction Nullifiers* to prevent a malicious proxy from replaying the same transaction with different funding notes. The Nullmask contract stores all Transaction Nullifiers and rejects transactions with previously used nullifiers. Each executed transaction on the EVM can be indexed by the unique `(chainId, address, nonce)` triple. Nullmask shielded actions mask this triple by hashing it together with the nullifying key.

definition 6: For a transaction `tx` and nullifying key `nk` from account `account`, Transaction Nullifier

$$\mathtt{nf}_{\mathtt{tx}} := \text{Poseidon2T4}\big(\mathtt{nk}, \mathtt{tx.chainId} \cdot 2^{32} + \mathtt{tx.nonce}, \mathtt{account}\big) \tag{1}$$

The chain ID and nonce are compacted into a single field element; therefore, the Transaction Nullifier requires only one Poseidon2T4 invocation.

6.2 Shielded Transfer Circuit

The Nullmask circuit verifies:

- Transaction Nullifier (Def. 6)
- Commitments of all funding notes (Alg. 5)
- Merkle tree paths of all funding notes
- Nullifiers of all funding notes (Def. 5)
- Shielded action balance (input amount = output amount)
- RLP deserialization of the virtual transaction
- ECDSA signature of the virtual transaction
- Virtual transaction sender matches the shielded action sender
- Virtual transaction recipient matches the shielded action recipient
- Virtual transaction gas fee equals the shielded action gas fee
- Change note correctness
- Merkle tree path of the recipient's key in the Key Registry
- New note encryption (Alg. 5.1)

7 Proxy Service

7.1 Local and Remote Deployment

The Nullmask proxy can run locally on the client side (mobile app or browser extension) or on a remote server. A locally running proxy provides full privacy but requires installation, which can present an obstacle to frictionless user onboarding.

Alternatively, the proxy can run on a remote server and be accessed via a webpage. This setup still guarantees on-chain privacy; however, the server operator can observe user transfers. This approach resembles Houdini private swaps [hou], where no party can trace on-chain swaps except the service operator.

7.2 User Onboarding

The onboarding procedure is straightforward:

1. (Optional) The user downloads the Nullmask app (mobile app or browser extension) and grants the necessary permissions.
2. The user opens the Nullmask app (or web app) and connects their wallet.
3. The app prompts the wallet to sign the login message.
4. The app prompts the wallet to add a new network.
5. The app enables the user to fund their shielded account.

From this point onward, everything is configured. The user fully controls their Nullmask shielded balance from their wallet. They can send transactions as usual and bridge their tokens between the Nullmask network and other networks.

7.3 Access Token

Unlike a traditional RPC node, where all data is public, the Nullmask RPC proxy must restrict data access to keep shielded account data private. We achieve this by embedding an access token in the RPC URL. Through this mechanism, the viewing key remains stored within the wallet.

7.4 Wallet-Only Mode

A unique feature of Nullmask is that it can be used directly from a wallet without opening a web3 application. The design challenge lies in the limited UI of the wallet. While the wallet exposes only a "send" action, Nullmask requires both "private send" and "private withdraw" actions. We resolve this one-to-two mapping as follows:

1. The user simply uses the "send" action.
2. The proxy checks whether the receiving address is registered on the Nullmask network.

3. If the address is registered, the proxy executes a "shielded send" to the address. Funds remain locked in the Nullmask contract.
4. If the address is not registered, the proxy executes a "shielded withdraw" to the address. Funds are released from the Nullmask contract to the address.

Owing to this logic, the user can execute both actions via a unified UI. As a side effect, users cannot withdraw (de-shield) to Nullmask-registered addresses. We consider this behavior a desirable privacy feature, as it encourages users to use fresh unlinkable addresses for withdrawals.

8 Key Registry

The mapping from 0x addresses to shielded addresses is stored on-chain in the Nullmask contract. Only an in-wallet-confirmed transaction can register or modify these mapping pairs. This approach eliminates any additional trust assumptions regarding the proxy or Nullmask services.

9 Private Voting and Airdrop Facilitation

The Nullmask key infrastructure can be leveraged to enhance the privacy of other services. For example, it can facilitate private voting and private airdrops. These protocols typically require pre-registration of eligible participants to assign them nullifying keys [AG22]. The PLUME signature [GG22] (ERC-7524) was proposed to eliminate this additional communication round, but it has not been widely adopted. Developers can bypass this setup phase by reusing Nullmask Public Nullifying Keys instead.

10 Hardware Wallets

Other privacy solutions (Railgun, 0xbow, Hinkal, zERC20, Zcash) rely on extracting the spending authority from the wallet. This makes them susceptible to various phishing attacks and violates the fundamental principle of hardware wallets: that the spending authority never leaves the device.

Railgun and Zcash addressed this by requiring an additional signature over the shielded transaction. However, both rely on custom hash functions (Zcash Orchard uses Sinsemilla; Railgun uses variable-rate Poseidon) that require large precomputed lookup tables, making them impractical for the constrained environment of a hardware wallet.

In contrast, Nullmask authorizes shielded transactions using standard EIP-1559 transactions, which are already implemented in every hardware wallet. Our design enables shielded transactions from any hardware wallet without any modifications to the wallet's code.

11 Compliance

To prevent illicit parties from laundering money through the Nullmask privacy pool, every deposit to the contract must be verified and approved. We employ the same mechanisms as Privacy Pools by 0xbow [BIN+23]: all pool deposits are inspected by chain analysis tools. If a depositing account is associated with illicit activity, the deposit is rejected and the user can only de-shield their funds back to the deposit address. Additionally, we expedite the approval process for funds originating from well-established centralized exchanges, since CEXs are responsible for rejecting illicit funds and collecting user KYC data.

Nullmask employs an additional novel mechanism that enables recovery of the pool even if some funds were approved to enter the pool and subsequently tainted retroactively. This is a key differentiator from Railgun, where no such mechanism exists. Since some tainted funds are already present in the Railgun pool and there is no way for users to disassociate from them, the entire pool remains tainted indefinitely.

The Nullmask guard publishes a revocation key with each approved deposit. Each Nullmask note includes encrypted information about the deposits that funded it. If a deposit is tainted retroactively, the guard publishes the secret key from the corresponding revocation key. This key enables each protocol participant to either disassociate from the tainted funds or—in the case that they hold tainted funds—to verifiably clean the rest of their untainted balance.

	Tornado Cash	Privacy Pools	Railgun	Nullmask
user interface	web app	web app	Railway wallet, terminal	any wallet
required backup of	every note	a new seed	a new seed	none
shielded transfers	✗	✗	✓	✓
shielded addresses	-	-	new type (0zk)	standard addresses (0x)
deposit amounts	fixed denominations	fixed minimum	any	any
ERC-20 token support	whitelist	whitelist	any	any
built-in compliance	none	exclusion proofs	PPoI	permissioned deposits and retroactive compliance
deployed	August 2019	March 2025	July 2021	January 2026
hardware wallets support	✗	✗	✓(flaw in design)	✓

Table 1: Comparison of privacy services in the Ethereum ecosystem.

12 Related Work

We compare Nullmask to several well-known privacy services in the Ethereum ecosystem in terms of user interface and user experience.

12.1 Tornado Cash

Tornado Cash uses a web app for deposits and withdrawals. For each deposit, the app generates a note plaintext that users must store securely in order to withdraw later.

12.2 Privacy Pools

Privacy Pools also uses a web app as its front end. Users must generate a new account backed up by a seed phrase. The seed phrase must be re-entered at every visit to the web app. Privacy Pools allows a single deposit to be withdrawn in multiple transactions to different addresses.

Update (March 17, 2026): Privacy Pools now supports key derivation from a wallet signature, accompanied by a prominent phishing warning disclaimer.

12.3 Railgun

Using Railgun requires the installation of a dedicated wallet backed up by a new seed phrase. Railgun users can transact privately among each other using a new address type prefixed with 0zk. Additionally, Railgun provides a terminal interface and a web app for 0x-to-0zk transfers.

13 Future Work

- **Unified Liquidity:** Employing a universal zk-bridge such as Boundless could enable our protocol to avoid fragmenting the privacy set across all the layers on which we deploy our smart contract.
- **Unified Nullifying Key Infrastructure:** Standardizing our key derivation scheme as an EIP (Ethereum Improvement Proposal) would enable all applications in the EVM ecosystem to use shielded addresses and services without user pre-registration.
- **Programmable Privacy:** Leveraging the power of recursive zk-SNARKs, we can allow users to deploy smart contracts with custom spending logic (e.g., multisig accounts, keypass recovery) within the Nullmask shielded privacy pool.
- **Minimizing Transaction Fees with IVC:** By leveraging Incrementally Verifiable Computation, we can defer on-chain proof verification until withdrawal. This reduces the gas cost of a fully shielded transaction by roughly 14×, from $\sim$ 1,400,000 (Railgun) to $\sim$ 100,000 gas per shielded transaction.

References

[AG22] Nalin Bhardwaj Aayush Gupta, Adhyyan Sekhsaria. Stealthdrops readme.md. https://github.com/stealthdrop/stealthdrop?tab=readme-ov-file#ecdsa-signature-verification, 2022.

[BIN+23] Vitalik Buterin, Jacob Illum, Matthias Nadler, Fabian Schär, and Ameen Soleimani. Blockchain privacy and regulatory compliance: Towards a practical equilibrium. September 2023. Available at SSRN.

[GG22] Aayush Gupta and Kobi Gurkan. PLUME: An ECDSA nullifier scheme for unique pseudonymity within zero knowledge proofs. Cryptology ePrint Archive, Paper 2022/1255, 2022.

[hou] Houdini swap: Swap bitcoin privately or any crypto anonymously. https://houdiniswap.com/. Accessed: 2025-09-24.